

产品需求文档 (Product Requirement)

产品名称 (Product Name)

Knowledge Foundry (知识熔炉)

产品概述 (Overview)

打造一款产品，将收集到的原始研究材料转化为由大语言模型（LLM）维护的知识库。系统能够将源文档摄入到原始知识存储中，将其编译成结构化的 Markdown Wiki，允许用户基于该 Wiki 提出复杂问题，并通过智能体（Agent）驱动的维护 workflow 持续增强知识库。

问题背景 (Problem)

目前，个人的研究工作流通常碎片化地分布在书签、PDF、笔记应用以及临时脚本中。高价值的上下文信息分散各处，难以查询，更难以随着时间推移而演进。现有的工具往往止步于“搜索”或单纯的“笔记记录”，而用户真正需要的是一个迭代式的工作流：收集资料、编译知识、提出问题、生成产出物，并将这些产出再次反馈闭环到系统中。

目标用户 (Users)

- 持续数周或数月追踪特定课题的独立研究人员
- 正在构建市场、技术或竞品知识库的创始人及产品团队
- 已经习惯使用 Markdown 并希望获得类似 Obsidian 般流畅体验的重度用户

产品目标 (Product Goal)

使用户只需将系统指向某一批原始来源资料，即可获得一个“活的”Markdown 知识库，该知识库能够：

- 自动组织概念并生成来源摘要
- 支持由 Agent 驱动的问答与知识综合
- 生成衍生性输出，如研究笔记、报告、幻灯片和可视化图表
- 通过健康检查和维护任务，随着时间的推移不断自我完善

核心用户旅程 (Core User Journey)

- 用户录入网页文章、论文、代码仓库、数据集、Markdown 笔记和图片等原始素材。
- 系统将这些原始资产存储在 `raw/` 目录下，并保留相关的元数据和溯源引用。
- LLM 编译流水线将这些语料库转换为结构化的 Markdown Wiki。
- 用户在一个类似 Obsidian 的文件工作区中探索该知识库。
- 用户可以提出问题，或请求系统生成报告、幻灯片等文档。
- Agent 会利用 Wiki 与原始材料去研究并回答该问题，最后将结果以 Markdown、幻灯片或图片的形式写回。
- 维护型 Agent 定期运行健康检查，找出文档矛盾点、失效链接、过时的摘要，并推荐候选的新文章。

系统作用域规范 (Scope)

- 将本地与网络来源文件摄入（Ingest）到一个托管语料库中

- 维护一个“Markdown 优先”的 Wiki 系统，支持反向链接、概念页面和来源摘要
- 暴露针对编译、问答和复盘类型工作流的编排层（Orchestration Layer）
- 提供一个 Web 应用端，用于浏览知识库并触发各项任务
- 提供后端 API 接口负责摄入、编译、搜索、查询、制品生成和健康检查
- 支持兼容 Obsidian 的文件格式，并将其作为第一公民的存储与导出格式
- 保证所有生成的输出均基于文件系统，便于随时检查

MVP 核心能力 (MVP Capabilities)

- **语料浏览器 (Corpus Browser)**：展示原始素材、已编译的 Wiki 页面以及生成的输出物
- **摄入任务 (Ingestion Job)**：注册原始文件并标准化元数据
- **编译任务 (Compile Job)**：基于原始内容创建或更新 Markdown Wiki 页面
- **搜索与问答流水线**：基于 Wiki 和支持性来源摘要进行检索与进行复杂问答
- **制品生成**：支持生成 Markdown 报告和 Marp 格式幻灯片
- **健康检查任务**：标记 Wiki 中的矛盾或不一致之处，并建议补充缺失页面/链接
- **/health 状态端点**：为系统编排与 UI 可视化提供系统状态反馈查询

非当前目标 (Non-Goals)

- 构建一个完整的实时协同在线编辑器
- 在 v1 版本即解决海量规模语料的分布式检索问题
- 在首个里程碑中训练或微调自定义的大语言模型
- 完全取代 Obsidian 成为用户的首选本地编辑环境

约束条件 (Constraints)

- Markdown 文件必须是核心规范化的知识输出格式
- 原始数据与衍生数据都必须保存在本地磁盘上供随时检查
- 系统在应对中小型语料库时应该良好运行，不能强制依赖十分沉重的 RAG 架构
- 自动生成的产出应该能够极易重新归档到 Wiki 系统中
- 系统架构应留有余地，方便日后扩展合成数据生成和微调（fine-tuning）流水线

成功标准 (Success Criteria)

- 用户在摄取一组原始材料后，不需要手动编辑文件就能获得一份概念连贯的 Markdown Wiki
- 用户能发起一个研究问题，并收到一份立足于现存知识库底座生成的 Markdown 或幻灯片产出
- 系统能够自动运行健康状态检查任务，并随着时间推移逐渐改善 Wiki 的内部结构及一致性
- 产出的项目结构对独立开发者或小型团队来说，必须保持极高的可读性与管理易用性

需求来源与灵感 (Requirement Sources & Inspirations)

1. Karpathy 的 LLM OS 架构

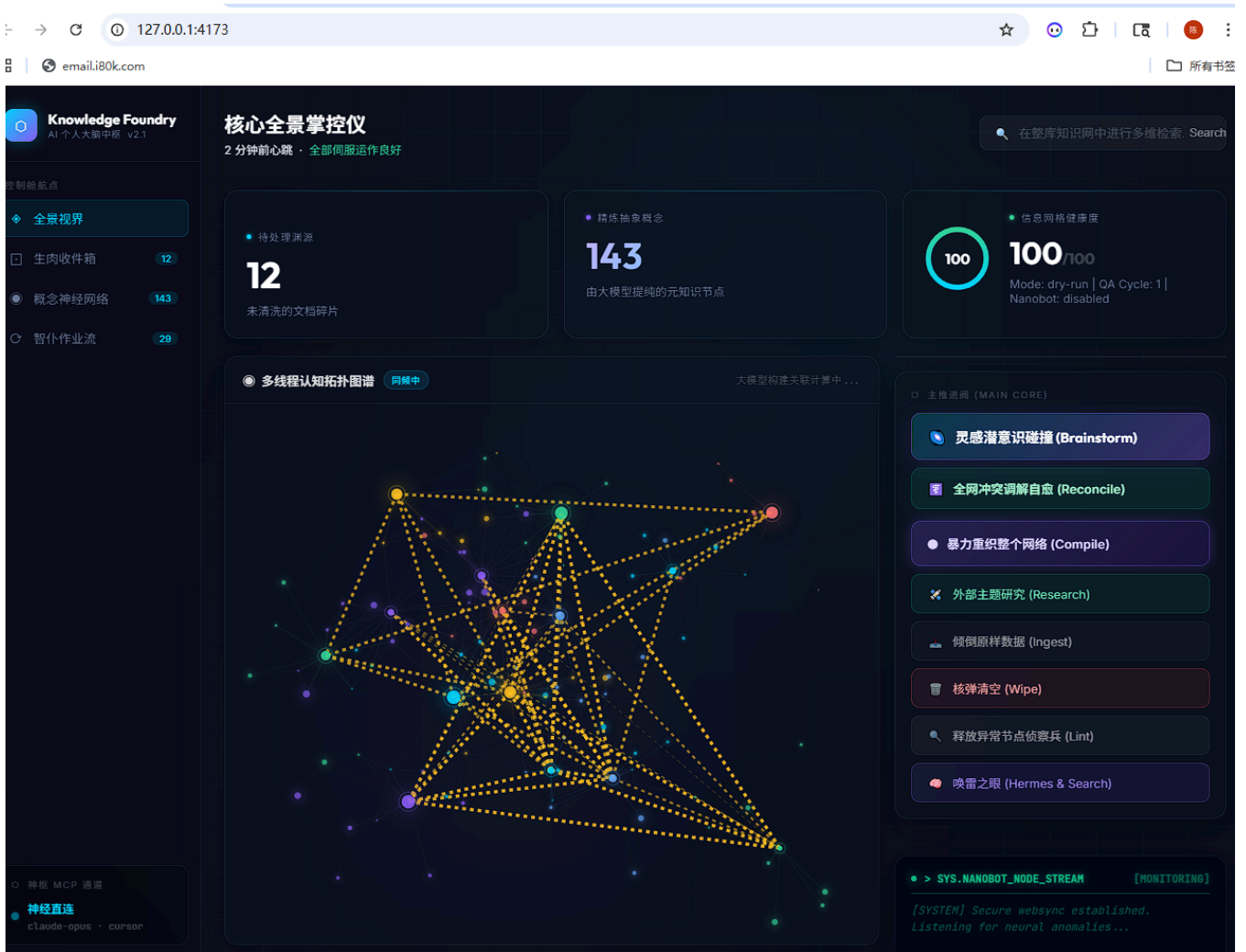
Knowledge Foundry 的核心哲学与架构设计深受 Andrej Karpathy 的 "LLM OS" (大语言模型操作系统) 概念启发。在这个理念下，计算机体系架构被中心化于 LLM 并得到了重新构想：

- **CPU (认知核心)**: 大语言模型 (LLM) 充当了核心推理引擎。与传统的执行确定性代码不同，它负责处理基于自然语言指令的任务。
- **RAM (上下文窗口)**: 模型在处理特定任务时用于持有即时信息的、容量受限的“工作空间内存”。
- **Storage (硬盘存储)**: 由本地 Markdown 文件系统和向量数据库 (即本项目中的 `data/wiki` 结构) 承载的长期记忆。
- **System Calls (工具使用)**: LLM 通过调用外部工具 (如 Python 解释器、网络爬虫、文件编译器) 作为“系统调用”，来统筹编排它的各种行为意图。
- **Peripherals (外设 I/O)**: 多模态输入输出能力 (如视觉感知、语音交互、网页操作等)。

Knowledge Foundry 通过将 Markdown 工作区视为一个标准的文件系统，并将游走在后台的各类 Agent (如 Nanobots 巡防兵、Hermes 搜捕哨等) 视为受此 LLM 内核调度的系统进程，将“大模型操作系统”这一理念映射并落地为了现实。

2. 仪表盘界面 (核心全景掌控仪)

此 LLM OS 在物理形态上，表现为一个具备中心控制能力的数字仪表盘面板，用以为系统提供针对 Agent 运行状态和知识图谱连接密度的全维度实时观测。



参考附图为当前产品的可视化编排视窗。定义了一套极具极客风格的“全景掌控仪”，提供支持多线程观测的认知流图谱计算面板、实时的深层思考节点连线动效，以及服务于系统级 Agent 工作流（如：灵感碰撞 Brainstorm、暴力重织全网 Compile、全局冲突解念 Reconcile)的任务启停中控台。